

מערכות הפעלה - שאלות תיאורטיות ופתרונות

תרגילים 1-3 | תרגום לעברית | ללא חלקי התכנות

הקובץ כולל רק את השאלות והפתרונות של **Part A - Theory** מתוך שלושת תרגילי הבית. שמות פונקציות, פקודות וקוד נשמרו באנגלית כדי לשמור על דיוק מקצועי.

תוכן עניינים

- תרגיל 1 - Threads וסנכרון
- תרגיל 2 - Kernel Mode, User Mode וסיגנלים
- תרגיל 3 - תזמון, זיכרון ומערכות קבצים

תרגיל 1 - Threads וסנכרון

מקור: ex1_sol(2).pdf

TQ1 - מקביליות מדומה לעומת מקביליות אמיתית

לתוכנית יש 3 תהליכונים, וכל אחד מהם זקוק ל-40 מילישניות של זמן CPU. אין פעולות קלט/פלט ואין חסימות.

1. בליבה יחידה, בהנחה שהמערכת עוברת בין תהליכונים מוכנים לריצה ועלות החלפת ההקשר זניחה, מהו זמן הקיר המינימלי עד שכל התהליכונים יסיימו?
 2. במעבד בעל 3 ליבות, תחת אותן הנחות, מהו זמן הקיר המינימלי?
 3. כל החלפת הקשר עולה 0.2 מילישניות. בליבה היחידה המערכת כופה החלפת הקשר לאחר כל 5 מילישניות של הרצת התהליכון הנוכחי. מהי התקורה הנוספת שנגרמת רק מהחלפות ההקשר עד לסיום?
- הניחו שמתבצעת החלפת הקשר גם לאחר מקטע ה-5 מילישניות האחרון שמסיים את התהליכון האחרון.

פתרון

1. בליבה יחידה רק תהליכון אחד יכול לרוץ בכל רגע.

$$3 \times 40 = 120 \text{ ms}$$

לכן זמן הקיר המינימלי הוא **120 מילישניות**. קיימת כאן concurrency, אך אין הרצה מקבילית אמיתית.

2. בשלוש ליבות ניתן להריץ את שלושת התהליכונים בו-זמנית. כל אחד זקוק ל-40

מילישניות, ולכן זמן הקיר המינימלי הוא **40 מילישניות**.

3. כל תהליכון מחולק ל-:

$$40 / 5 = 8 \text{ מקטעים}$$

בסך הכול:

$$3 \times 8 = 24 \text{ החלפות הקשר}$$

ולכן התקורה היא:

$$24 \times 0.2 = 4.8 \text{ ms}$$

התקורה הכוללת: 4.8 מילישניות.

TQ2 - שזירות אפשריות בין תהליכונים

המשתנה הגלובלי המשותף x מתחיל בערך 0. שני תהליכונים, A ו-B, מריצים פעם אחת כל אחד את הקוד הבא:

```
t = x
t = t + 1
x = t
```

הניחו כי t הוא משתנה מקומי לכל תהליכון.

1. אילו ערכים סופיים של x אפשריים?
2. הציגו שזירה מפורשת שמובילה לכל אחד מהערכים האפשריים.

פתרון

הערכים האפשריים הם 1 או 2.

שזירה שמובילה ל- $x = 2$

```
A: tA = x      // tA = 0
A: tA = tA + 1  // tA = 1
A: x = tA      // x = 1
B: tB = x      // tB = 1
B: tB = tB + 1  // tB = 2
B: x = tB      // x = 2
```

שזירה שמובילה ל- $x = 1$

```
A: tA = x      // tA = 0
B: tB = x      // tB = 0; 0 שני התהליכונים קראו
A: tA = tA + 1  // tA = 1
B: tB = tB + 1  // tB = 1
A: x = tA      // x = 1
B: x = tB      // x 1 נשאר
```

במקרה השני מתרחש **Lost Update**: שתי פעולות ההגדלה התבססו על אותו ערך התחלתי.

TQ3 - מנעול TAS לעומת Ticket Lock

נתונים שני מנעולים:

- מנעול סיבוב מסוג Test-and-Set (TAS).
- Ticket Lock: כל תהליכון מקבל $\text{my_ticket} = \text{fetch_add}(\text{ticket}, 1)$ וממתין עד ש- $\text{cur_ticket} == \text{my_ticket}$.

1. איזה מנעול מבטיח סדר כניסה FIFO? הוכיחו.
2. איזה מנעול מבטיח זמן המתנה חסום, ללא starvation, בהנחה שכל תהליכון שנכנס למנעול משחרר אותו לבסוף? הוכיחו.
3. הסבירו בקצרה מדוע מנעול TAS עלול לגרום ל-starvation.

פתרון

1. **Ticket Lock מבטיח FIFO.** כל תהליכון מקבל באופן אטומי מספר כרטיס ייחודי וגדל. הכניסה מותרת רק כאשר cur_ticket שווה למספר הכרטיס של התהליכון. בכל שחרור cur_ticket גדל בדיוק ב-1. לכן הכרטיסים מטופלים בסדר עולה, וזהו סדר FIFO.
2. **Ticket Lock מבטיח המתנה חסומה.** נניח שתהליכון קיבל כרטיס k , ובאותו רגע $\text{cur_ticket} = c$. לפניו נמצאים בדיוק $k - c$ כרטיסים. בכל שחרור מגדילים את cur_ticket ב-1. לכן, בהנחה שכל מחזיק משחרר את המנעול, התהליכון ייכנס לאחר לכל היותר $k - c$ שחרורים. מכאן שאין starvation.
3. ב-TAS כל שחרור יוצר מרוץ חדש בין כל הממתנים. תהליכון בעל תזמון לא מוצלח יכול להפסיד שוב ושוב ללא גבול, משום שאין תור או כלל סדר שמבטיח שהוא יהיה הזוכה הבא.

TQ4 - בעיית היצרן והצרכן

יש חוצץ חסום בקיבולת CAP, המשותף למספר יצרנים וצרכנים. משתמשים בשלושה אובייקטים לסנכרון:

- mutex - מניעה הדדית בגישה לחוצץ.
- empty - סמפור חסום בטווח $[0, CAP]$, מאותחל ל-0.
- full - סמפור חסום בטווח $[0, CAP]$, מאותחל ל-0.

יצרן - גרסה שגויה

1. acquire mutex
2. wait(empty)
3. insert one item
4. signal(full)
5. release mutex

צרכן - גרסה שגויה

1. acquire mutex
2. wait(full)
3. remove one item
4. signal(empty)
5. release mutex

1. הסבירו כיצד האלגוריתם עלול להיכנס ל-deadlock. יש להציג תרחיש קונקרטי: מי מחזיק במה ומי ממתין למה.

2. תנו תיקון מינימלי והסבירו מדוע הוא מונע deadlock.

פתרון

תרחיש deadlock:

1. החוצץ ריק: $full = 0$ ו- $empty = CAP$.
2. צרכן נועל את mutex.
3. הצרכן מפעיל $wait(full)$. מאחר ש- $full = 0$, הוא נחסם אך ממשיך להחזיק ב-mutex.
4. יצרן מנסה לנעול את mutex, אך נחסם.

הצרכן ממתין לכך שיצרן יכניס פריט ויבצע $signal(full)$, אך היצרן אינו יכול להיכנס לחלק הקריטי, משום שהצרכן החסום מחזיק ב-mutex. זוהי המתנה מעגלית.

התיקון המינימלי

יש להעביר את פעולות wait לפני נעילת ה- mutex .

יצרן מתוקן

1. wait(empty)
2. acquire mutex
3. insert one item
4. release mutex
5. signal(full)

צרכן מתוקן

1. wait(full)
2. acquire mutex
3. remove one item
4. release mutex
5. signal(empty)

כעת תהליכון יכול להיחסם על סמפור, אך הוא נחסם בלי להחזיק ב- mutex . לכן הצד השני יכול להיכנס לחלק הקריטי, לבצע התקדמות ולשחרר את הסמפור הדרוש. בכך מוסר תנאי ההמתנה המעגלית.

תרגיל 2 - Kernel Mode, User Mode וסיגנלים

מקור: ex2_sols(2).pdf

TQ1 - חישוב עלות: int 0x80 לעומת syscall

תוכנית במרחב המשתמש מבצעת $N = 10,000$ קריאות מערכת `write()` בלולאה צפופה.

נשווה בין שתי דרכים למעבר מ-User Mode ל-Kernel Mode:

- `int 0x80` - המנגנון הישן, המשתמש במסלול כללי של פסיקת תוכנה.
- `syscall` - פקודת CPU ייעודית לקריאות מערכת.

בשני המקרים אותו תהליכון ממשיך לרוץ בקוד הקרנל. המעבר כשלעצמו אינו החלפת הקשר לתהליכון או לתהליך אחר.

עלויות משותפות לכל קריאת מערכת

עלות במחזורי שעות	שלב
250	כניסה ל-Kernel Mode ומעבר למצב הרצה של הקרנל
50	<code>syscall_dispatch</code> - בחירת שירות הקרנל לפי מספר קריאת המערכת
2,000	עבודת הקרנל בפועל - <code>sys_write</code>
200	חזרה מ-Kernel Mode ל-User Mode

עלות נוספת במסלול int 0x80

עלות	שלב
80	חיפוש ב-IDT עבור פסיקת תוכנה 0x80
220	מסלול כניסה כללי של <code>interrupt/trap</code> : שמירת מצב מלאה, זיהוי סיבת ה- <code>trap</code> והגעה ל- <code>dispatch</code>

עלות נוספת במסלול syscall

עלות	שלב
30	קפיצה לכתובת הכניסה המוגדרת ב- <code>MSR_LSTAR</code>
70	מסלול ייעודי: שמירת מצב מינימלית והכנה ל- <code>dispatch</code>

1. חשבו את העלות הכוללת לקריאת מערכת בכל מסלול.
2. חשבו את מספר המחזורים לכל $10,000$ הקריאות ואת החיסכון המוחלט של `syscall`.
3. חשבו את אחוז התקורה בכל מסלול. תקורה מוגדרת ככל העלות שאינה `2,000` המחזורים של `sys_write`.

4. נקודת איזון: במערכת צעצוע, תמיכה ב- `syscall` דורשת עלות חד-פעמית של $C = 10,000$ מחזורים. מהו מספר הקריאות המינימלי שעבורו `syscall` זול ממש מ- `int 0x80`?

פתרון

1. העלות המשותפת:

$$250 + 50 + 2,000 + 200 = 2,500$$

מסלול `int 0x80` מוסיף:

$$80 + 220 = 300$$

ולכן:

$$2,500 + 300 = 2,800 \text{ cycles}$$

מסלול `syscall` מוסיף:

$$30 + 70 = 100$$

ולכן:

$$2,500 + 100 = 2,600 \text{ cycles}$$

`int 0x80`: 2,800 מחזורים; `syscall`: 2,600 מחזורים.

$$2,800 \times 10,000 = 28,000,000 \quad .2$$

$$2,600 \times 10,000 = 26,000,000$$

החיסכון:

$$28,000,000 - 26,000,000 = 2,000,000$$

כלומר **2,000,000 מחזורי שעות**.

3. במסלול הישן:

$$\text{overhead} = 2,800 - 2,000 = 800$$

$$800 / 2,800 \approx 28.6\%$$

במסלול המודרני:

$$\text{overhead} = 2,600 - 2,000 = 600$$

$$600 / 2,600 \approx 23.1\%$$

4. העלות הישנה: $2,800N$. העלות המודרנית: $10,000 + 2,600N$.

$$10,000 + 2,600N < 2,800N$$

$$10,000 < 200N$$

$$N > 50$$

לכן מספר הקריאות המינימלי הוא **51**. עבור 50 קריאות העלויות שוות: 140,000 מחזורים.

TQ2 - מנגנוני כניסה לקרנל ומסירת סיגנלים

נבחנים כמה מנגנונים למעבר או תקשורת בין User Mode ל-Kernel Mode:

- **System call** - התוכנית מבקשת במכוון שירות ממערכת ההפעלה.
- **Exception** - המעבד מזהה פעולה לא חוקית או בלתי צפויה.
- **Software interrupt** - התוכנית מבצעת במפורש `int X`.
- **Signal** - מערכת ההפעלה מודיעה לתהליך או לתהליכון על אירוע.

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

void handler(int sig)
{
    printf("handler\n");
}

void* T1_main(void* arg)
{
    signal(SIGUSR1, handler);

    write(STDOUT_FILENO, "A", 1);    // E1
    __asm__ volatile("int $3");      // E2
    int x = 1 / 0;                    // E3

    return NULL;
}

void* T2_main(void* arg)
{
    sigset_t set;
    int sig;

    sigemptyset(&set);
    sigaddset(&set, SIGUSR1);

    pthread_sigmask(SIG_BLOCK, &set, NULL); // E4
    sigwait(&set, &sig);                      // E5

    return NULL;
}
```

הנחות עיקריות:

- המערכת היא Linux על x86-64.
 - write() משתמש לבסוף בפקודת syscall.
 - חלוקה באפס ב-E3 גורמת לחריגת CPU המדווחת לתהליך באמצעות SIGFPE.
 - לא הוגדר handler מותאם ל-SIGFPE.
 - תהליך חיצוני שולח SIGUSR1 כאשר T2 כבר ממתין בתוך sigwait().
 - לפי המודל מההרצאה, תהליכון שממתין ב-sigwait() מקבל את הסיגנל לפני מסירה ל-handler רגיל.
1. סווגו את E1-E5 כאחד מאלה: system call, software interrupt, exception, signal masking, signal waiting.
2. עבור E1-E3 קבעו האם המעבד נכנס ל-Kernel Mode, האם מדובר בהכרח בהחלפת הקשר, ומה הסיבה לכניסה לקרנל.
3. מי מקבל את T1 : SIGUSR1 באמצעות handler, או T2 באמצעות sigwait? נמקו.
4. קבעו אמת או שקר ותקנו את המשפטים השגויים:
- i. קריאת מערכת היא החלפת הקשר מתהליך המשתמש לתהליך קרנל.
 - ii. 3 int היא קריאת מערכת משום שהיא משתמשת בפקודת int.
 - iii. ה-handler חייב לרוץ באותו תהליכון שקרא ל-signal().
 - iv. sigwait ממתינה לסיגנל בלי להריץ את ה-handler.
 - v. אם לא הוגדר handler לסיגנל כמו SIGFPE, מערכת ההפעלה מפעילה את ברירת המחדל שלו.

פתרון

אירוע	סיווג	הסבר
E1	System call	התוכנית מבקשת ממערכת ההפעלה לכתוב בית לפלט התקני.
E2	Software interrupt	3 int מפעילה פסיקת תוכנה, המשמשת בדרך כלל לנקודת עצירה.
E3	Exception	חלוקה באפס היא פעולה בלתי חוקית שה-CPU מזהה.
E4	Signal masking	התהליכון משנה את מסכת הסיגנלים שלו וחוסם מסירה רגילה של SIGUSR1.
E5	Signal waiting	התהליכון ממתין באופן סינכרוני לאחד הסיגנלים החסומים בקבוצה.

אירוע	כניסה ל-Kernel Mode? ?Mode	בהכרח Context Switch? ?Switch	סיבת הכניסה
E1	כן	לא	בקשת שירות ממערכת ההפעלה: כתיבה לפלט.
E2	כן	לא	הפעלה מפורשת של פסיקת תוכנה.
E3	כן	לא	המעבד זיהה פעולה לא חוקית: חלוקה באפס.

כניסה ל-Kernel Mode פירושה שאותו תהליכון ממשיך לרוץ, אך כעת מבצע קוד קרנל. Context switch הוא מעבר להרצת תהליכון או תהליך אחר. האירוע עלול להוביל בהמשך לחסימה, לתזמון או לסיום התהליך, אך הכניסה עצמה אינה בהכרח החלפת הקשר.

T2 מקבל את SIGUSR1 באמצעות sigwait. הסיגנל נצרך סינכרונית: sigwait חוזרת, המשתנה sig מקבל SIGUSR1, וה-handler אינו מופעל עבור הסיגנל הזה.

- i. **שקר.** קריאת מערכת היא מעבר של אותו תהליכון מ-User Mode ל-Kernel Mode, לא מעבר ל"תהליך קרנל".
- ii. **שקר.** 3 int היא פסיקת תוכנה. לא כל פקודת X int היא קריאת מערכת.
- iii. **שקר.** סיגנל שמופנה לתהליך יכול להימסר לכל תהליכון כשיר בתהליך.
- iv. **אמת.** לפי המודל הנתון, sigwait צורכת את הסיגנל בלי להריץ handler רגיל.
- v. **אמת.** ללא handler מותאם מופעלת פעולת ברירת המחדל. עבור SIGFPE זו בדרך כלל סיום התהליך.

TQ3 - מסירת סיגנל בזמן החזקת Mutex

```

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

// Thread A
void* incrementer(void* arg) {
    // A1
    pthread_mutex_lock(&lock);
    counter++;
    pthread_mutex_unlock(&lock);
    // A2
    return NULL;
}

// Thread B
void* reader(void* arg) {
    int v;
    // B1
    pthread_mutex_lock(&lock);
    v = counter;
    pthread_mutex_unlock(&lock);
    // B2
    printf("%d\n", v);
    return NULL;
}

// SIGUSR1 handler
void handler(int sig) {
    pthread_mutex_lock(&lock);
    counter--;
    pthread_mutex_unlock(&lock);
}

```

ה-handler אינו בטוח לשימוש אמיתי, אך יש לנתח את הקוד כפי שהוא. הסיגנל נשלח פעם אחת, ויכול להימסר ל-A או ל-B. ה-mutex אינו רקורסיבי: תהליכון שמנסה לנעול שוב mutex שכבר נמצא בבעלותו ייחסם לנצח.

1. בהנחה שה-handler אינו קוטע חלק קריטי באמצע, השלימו את התוצאות לכל סדר של A, B ו-H. A מגדיל, B קורא ומדפיס, H מקטין.
2. הסבירו בדיוק מתי עלול להתרחש deadlock לאחר שמאפשרים לסיגנל להגיע בזמן חלק קריטי.

3. מנעו את ה-deadlock באמצעות חסימה זמנית של SIGUSR1, בלי לשנות את ה-handler ובלי לחסום את הסיגנל לנצח. ציינו היכן להשתמש בסמנים A1, A2, B1 ו-B2.

פתרון

#	סדר החלקים הקריטיים	הערך ש-B מדפיס	הערך הסופי של counter
1	$A \rightarrow B \rightarrow H$	1	0
2	$A \rightarrow H \rightarrow B$	0	0
3	$B \rightarrow A \rightarrow H$	0	0
4	$B \rightarrow H \rightarrow A$	0	0
5	$H \rightarrow A \rightarrow B$	0	0
6	$H \rightarrow B \rightarrow A$	1-	0

הערך הסופי תמיד 0, משום ש-A מוסיף 1 ו-H מחסיר 1. ההדפסה תלויה במועד הקריאה של B.

מתי מתרחש deadlock?

deadlock מתרחש כאשר SIGUSR1 נמסר לתהליכון שכבר מחזיק ב-lock :

- ל-A לאחר הנעילה ולפני השחרור, או
- ל-B לאחר הנעילה ולפני השחרור.

ה-handler רץ בתוך אותו תהליכון ומנסה לבצע שוב `pthread_mutex_lock(&lock)`. מכיוון שה-mutex אינו רקורסיבי, התהליכון נחסם. הוא אינו יכול לחזור מה-handler אל הקוד המקורי שישחרר את המנעול, ולכן התוכנית נתקעת.

מניעה באמצעות מסכת סיגנלים

תהליכון	חסימת SIGUSR1	שחזור המסכה הקודמת
A	ב-A1, לפני נעילת ה-mutex	ב-A2, לאחר שחרור ה-mutex
B	ב-B1, לפני נעילת ה-mutex	ב-B2, לאחר שחרור ה-mutex

יש לחסום בשני התהליכונים, משום שהסיגנל יכול להימסר לכל אחד מהם. החסימה מונעת הרצת handler באותו תהליכון בזמן שהוא עלול להחזיק במנעול. לאחר השחרור משחזרים את המסכה, ולכן הסיגנל אינו חסום לצמיתות.

TQ4 - החלפת תהליכוני משתמש באמצעות sigsetjmp / siglongjmp

בתרגיל זה **uthread** הוא תהליכון משתמש המנוהל על ידי התוכנית באמצעות הקשרים שמורים, ולא POSIX thread נפרד. שני ה-threads רצים על POSIX thread יחיד.

```
#define STACK_SIZE 4096

sigjmp_buf env[2];
char stack0[STACK_SIZE];
char stack1[STACK_SIZE];

int current_uthread = -1;
int done[2] = {0, 0};

void setup_context(int tid, char *stack, thread_entry_point entr,
int uthread_terminate(int tid);

/* גרסה שגויה */
void switch_to(int tid)
{
    siglongjmp(env[tid], 1);
    current_uthread = tid;
}

void yield(void)
{
    int me = current_uthread;
    int other = 1 - me;

    if (sigsetjmp(env[me], 1) == 0) {
        switch_to(other);
    }
}

void uthread_here(void)
{
    printf("I'm here\n");
    yield();

    printf("I'm here again\n");
    done[0] = 1;
    uthread_terminate(current_uthread);
}
```

```

}

void uthread_here(void)
{
    printf("I'm there\n");
    yield();

    printf("I'm there again\n");
    done[1] = 1;
    uthread_terminate(current_uthread);
}

int main(void)
{
    setup_context(0, stack0, uthread_here);
    setup_context(1, stack1, uthread_here);
    switch_to(0);
    return 0;
}

```

1. מצאו והסבירו את הבאג ב- `switch_to`, כולל השפעת `siglongjmp` ומדוע `yield()` עלולה לשמור הקשר במיקום שגוי.
2. כתבו גרסה מתוקנת של `switch_to`.
3. בהנחה שהבאג תוקן, כתבו את פלט התוכנית עד ששני ה-`uthreads` מסתיימים.
4. הסבירו מה קורה ב- `yield()` כאשר `sigsetjmp` מחזירה 0 וכאשר היא מחזירה ערך שאינו 0.
5. האם מחשב בעל שתי ליבות יכול להאיץ את ה-`uthreads` באמצעות מקביליות אמיתית? נמקו.

פתרון

1. הבאג הוא ש- `current_uthread` מתעדכן **לאחר** הקריאה ל- `siglongjmp`. הקריאה ל- `siglongjmp` אינה חוזרת לשורה הבאה ב- `switch_to`; היא משחזרת את `env[tid]` וממשיכה מן ה- `sigsetjmp` המתאים או מההקשר שהוכן על ידי `setup_context`. לכן ההשמה כלל אינה מתבצעת בזמן המעבר.
- כתוצאה מכך `current_uthread` נשאר מיושן. לדוגמה, 1 `uthread` עשוי לרוץ כאשר המשתנה עדיין שווה 0. כאשר הוא קורא ל- `yield()`, הוא מחשב `me = 0` ושומר את ההקשר של 1 `uthread` בתוך `env[0]`, ובכך דורס את ההקשר של 0 `uthread`.

```

void switch_to(int tid)
{
    current_uthread = tid;
}

```

```
    siglongjmp(env[tid], 1);  
}
```

3. I'm here
I'm there
I'm here again
I'm there again

4. כאשר `sigsetjmp(env[me], 1)` מחזירה 0, זו החזרה הישירה לאחר שמירת ההקשר הנוכחי. לכן קוראים ל- `switch_to(other)` ועוברים ל-`uthread` האחר.

כאשר היא מחזירה ערך שאינו 0, ה-`uthread` חודש באמצעות `siglongjmp`. גוף ה-`if` אינו מתבצע, `yield()` חוזרת, וה-`uthread` ממשיך אחרי הנקודה שבה ויתר על ה-CPU.

5. לא. שני ה-`uthreads` רצים בתוך `POSIX thread` יחיד. הפונקציות `sigsetjmp` ו-`siglongjmp` רק שומרות ומשחזרות הקשרים באותו `thread`; הן אינן יוצרות `kernel thread` נוסף. לכן רק `uthread` אחד רץ בכל רגע, גם אם קיימות כמה ליבות.

תרגיל 3 - תזמון, זיכרון ומערכות קבצים

מקור: ex3_sols(2).pdf

TQ1 - גרסה של מתזמן MLFQ

נתון מתזמן Multi-Level Feedback Queue עם שלוש רמות עדיפות:

Quantum	תור
ms 10	Q0
ms 20	Q1
ms 40	Q2

כללים:

- תהליכון חדש מתחיל ב-Q0, העדיפות הגבוהה ביותר.
- תהליכון שמנצל את מלוא ה-quantum יורד רמה אחת. תהליכון ב-Q2 נשאר שם.
- תהליכון שמבצע yield() או נחסם לפני סיום ה-quantum נשאר ברמתו.
- המתזמן בוחר תמיד מהתור הלא-ריק בעל העדיפות הגבוהה ביותר. בתוך תור משתמשים ב-FIFO.

שני תהליכונים מגיעים בזמן 0, וכל אחד זקוק ל-1,000 ms של CPU:

- A הוא CPU-bound אמיתי ורץ עד שהוא מופסק.
- B גם הוא CPU-bound, אך מתחזה ל-I/O-bound ומבצע yield() לאחר 9.9 ms בכל quantum.

1. לאחר ההגעה למצב יציב, באיזה תור נמצא כל תהליכון?
2. כמה מלאים A משלים לכל היותר לפני ש-B משתלט על ה-CPU? איזה אחוז מה-CPU מקבל A במצב היציב כל עוד שניהם חיים?
3. מהי ההנחה השגויה של גרסת MLFQ זו לגבי B?
4. האם B יכול אי פעם לרדת רמת עדיפות לפי הכללים הנתונים?

פתרון

1. **A נמצא ב-Q1 ו-B נמצא ב-Q0.** מנצל quantum מלא של 10 ms ב-Q0 ויורד ל-Q1. B מוותר לאחר 9.9 ms ולכן לעולם אינו עוזב את Q0.
2. A משלים בדיוק quantum מלא אחד של 10 ms בתחילת הריצה. לאחר מכן, כל עוד B חי, A מקבל **0% מה-CPU**, מפני שהמתזמן תמיד מוצא את B מוקן בתור Q0, בעל העדיפות הגבוהה יותר.
3. המתזמן מניח שכל תהליכון שמוותר מרצונו על ה-CPU הוא תהליכון אינטראקטיבי או I/O-bound הראוי לעדיפות גבוהה, אף ש-B אינו מבצע I/O ואינו נחסם באמת.

4. לא. הירידה מתרחשת רק לאחר ניצול quantum מלא, ו-B תמיד מוותר לפני סיומו.
המתזמן גם אינו עוקב אחר צריכת CPU מצטברת.

TQ2 - Demand Paging והחלפת דפים

גודל דף הוא 4 KiB, הזיכרון עובד בשיטת demand paging, ולתהליך זמינות 3 מסגרות פיזיות בלבד. בכל PTE קיימים valid bit, reference bit ו-dirty bit.

מחרוזת הגישות:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

כללי Clock:

- שלושת ה-faults הראשונים ממלאים את 0, 1 Frame ו-2 לפי הסדר.
 - דף חדש מקבל 1 = reference bit.
 - ב-hit קובעים 1 = reference bit.
 - ב-fault כשהמסגרות מלאות, מתחילים במיקום המחוג. דף עם 1 bit מקבל 0 והמחוג מתקדם; הדף הראשון בעל 0 bit מפונה. הדף החדש נטען עם 1 bit, והמחוג מתקדם למיקום הבא.
1. השלימו: page fault מתרחש כאשר valid bit של הדף הוא _____. מערכת ההפעלה טוענת אותו לתוך ____.
2. דמו את המחרוזת באמצעות LRU ומצאו את מספר ה-page faults.
3. דמו את אותה מחרוזת באמצעות Clock ומצאו את מספר ה-page faults.
4. LRU הניב ____ faults ו-Clock הניב ____ faults. אמת או שקר: מדיניות מקורבת כמו Clock לעולם אינה יכולה לגרום לפחות faults מ-LRU מדויק.
5. איזה דף דורש כתיבה נוספת ל-swap לפני שימוש חוזר במסגרת: clean או dirty? נמקו.

פתרון

1. **valid bit = 0**. מערכת ההפעלה טוענת את הדף לתוך **מסגרת פנויה**; אם אין מסגרת פנויה, היא מפנה דף קורבן תחילה.
2. LRU גורם ל-10 **page faults**.

גישה	F/H	דפים תושבים לאחר הגישה
1	F	{1}
2	F	{1,2}
3	F	{1,2,3}
4	F	{2,3,4}
1	F	{1,3,4}

גישה	F/H	דפים תושבים לאחר הגישה
2	F	{1,2,4}
5	F	{1,2,5}
1	H	{1,2,5}
2	H	{1,2,5}
3	F	{1,2,3}
4	F	{2,3,4}
5	F	{3,4,5}

3. Clock גורם ל-9 page faults.

גישה	F/H	Frame 0	Frame 1	Frame 2	Hand
1	F	(1,1)	—	—	1
2	F	(1,1)	(2,1)	—	2
3	F	(1,1)	(2,1)	(3,1)	0
4	F	(4,1)	(2,0)	(3,0)	1
1	F	(4,1)	(1,1)	(3,0)	2
2	F	(4,1)	(1,1)	(2,1)	0
5	F	(5,1)	(1,0)	(2,0)	1
1	H	(5,1)	(1,1)	(2,0)	1
2	H	(5,1)	(1,1)	(2,1)	1
3	F	(5,0)	(3,1)	(2,0)	2
4	F	(5,0)	(3,1)	(4,1)	0
5	H	(5,1)	(3,1)	(4,1)	0

4. LRU הניב 10 ו-Clock הניב 9. הטענה **שקרית**. Clock רק מעריך עדכניות באמצעות ביט יחיד, ולכן בחירת הקורבן יכולה להיות שונה מ-LRU ולעיתים במקרה להיות טובה יותר. מקורב פירושו לא מדויק, לא בהכרח גרוע תמיד.

5. **Dirty page**. תוכנו שונה מהעותק בדיסק או ב־swap, ולכן יש לכתוב אותו חזרה לפני שימוש חוזר במסגרת. clean page כבר תואם לעותק המאוחסן וניתן פשוט להשליך אותו.

TQ3 - קישורים, מחיצות ובלוקים עקיפים

במחשב Linux של Alice קיימות שתי מחיצות ext4 נפרדות: אחת מותקנת ב- /home והשנייה ב- /data. קיים קובץ בגודל 5 GiB בנתיב /home/alice/report.pdf, וכן hard link אליו בנתיב /home/alice/backup/report.pdf.

שתי הפקודות הן חלופות עצמאיות, וכל אחת מתחילה מהמצב המקורי:

Command A: `mv /home/alice/report.pdf /home/alice/archive/report.pdf`
 Command B: `mv /home/alice/report.pdf /data/report.pdf`

1. כמה בתי תוכן מועתקים בפקודה A וכמה בפקודה B? מה פקודה A משנה בפועל בדיסק?
2. לאחר פקודה B, כמה hard links נשארים ל-inode המקורי במחיצת /home? מה תציג הפקודה `cat /home/alice/backup/report.pdf`?
3. אמת או שקר: hard link יכול להפנות לקובץ שה-inode שלו נמצא במחיצה אחרת. מדוע הפקודה `ln /home/alice/backup/report.pdf /data/hardlink.pdf` נכשלת?
4. נניח שה-backup היה symbolic link לנתיב המקורי. מה תציג פקודת `cat` לאחר פקודה B?
5. ב- /data מבנה inode כולל 12 מצביעים ישירים, מצביע עקיף יחיד, כפול ומשולש. גודל בלוק 4 KiB וגודל מצביע 4 בתים. עבור קובץ בגודל 5 GiB:
 1. כמה מצביעים נכנסים בבלוק עקיף אחד?
 2. כמה בלוקי נתונים תופס הקובץ?
 3. לכמה בלוקי נתונים ניתן להגיע באמצעות direct + single-indirect + double-indirect?
 4. האם נדרש triple-indirect?

פתרון

1. פקודה A מעתיקה **0 בתים**. פקודה B מעתיקה **5 GiB**. מעבר בתוך אותה מחיצה מעדכן רק רשומות ספרייה; ה-inode ובלוקי הנתונים נשארים ללא שינוי. מעבר בין מחיצות דורש העתקת התוכן למחיצה החדשה ומחיקת הרשומה הישנה.
2. נשאר **hard link אחד**. פקודת `cat` מציגה את **התוכן המקורי**, משום שהנתונים נמחקים רק לאחר שכל רשומות הספרייה המפנות ל-inode נמחקו.
3. **שקר**. מספר inode משמעותי רק בתוך מערכת הקבצים או המחיצה שלו. רשומת ספרייה ב- /data אינה יכולה להפנות ל-inode של /home, ולכן יצירת hard link בין המחיצות נכשלת.
4. תתקבל **שגיאת broken link**. symbolic link שומר נתיב, ופקודה B הסירה את הנתיב /home/alice/report.pdf; לכן הקישור מצביע ליעד שאינו קיים.

$$4096 / 4 = 1024 \text{ pointers} \quad .1 \quad .5$$

$$5 \text{ GiB} / 4 \text{ KiB} = 5 \times 2^{30} / 2^{12} = 1,310,720 \text{ data blocks} \quad .2$$

$$12 + 1024 + 1024 \times 1024 = 1,049,612 \text{ data blocks} \quad .3$$

4. כן.

$$1,310,720 > 1,049,612$$

לכן direct + single + double אינם מספיקים ונדרש מצביע triple-indirect.